INCREMENTAL REPOSITORY REVIEW

Asymptotic

Exact-root admission after the numerical precision guard

Source analysis, a proof-preserving repair candidate,
and independently executed regression models

Repository	`VladimirReshetnikov/Asymptotic`
Reviewed revision	`8e859961d7d37f008b826f3a8cad406460271614`
Review date	September 10, 2026
Novelty baseline	Four review waves; 36 packages; 231 attributed entries
Retained additional item	One conservative coverage limitation, not a demonstrated wrong mathematical result
Executed evidence	30 independent Python test methods, with deterministic nested families
Not executed	The repository, the WL candidate, or package regressions in Wolfram/Mathics

---

Principal result. The newly installed Mathics numerical safeguard can recognize an unchanged integer seed as an exact polynomial root, but not an equally verifiable noninteger dyadic seed. Consequently, an exact inverse such as $x = y$ can be refused at $y = 1/2$ under the default 50-digit numerical-check request. This is a source-derived prediction with supplied fresh-kernel probes, not a recorded package execution.

Review discipline. Existing issues are not counted again. The report distinguishes a newly narrowed acceptance boundary from the earlier unsafe precision claim that the safeguard addresses. Apparent certificate and powered-observable issues were screened out when they were already covered or did not contradict the stated contract. Prepared for Vladimir Reshetnikov. The archive contains this article,

candidate WL code, independent exact-arithmetic code, tests, evidence, and a novelty/coverage ledger.

Contents

1 Executive verdict

The useful outcome of a fifth incremental review is not another catalog of familiar defects. The repository already maintains an unusually extensive review inventory. Its current index records 36 packages and 231 attributed entries before overlap consolidation, and explicitly distinguishes historical observations from current acceptance. This review used that inventory, the maintained status register, both later-wave intakes, and relevant original material to avoid presenting old work as new.[1, 2, 3, 4]

One additional item is retained: N01, the exact-rational seed coverage gap in the new Mathics numerical admission helper. Its severity is medium for compatibility and user experience, low for mathematical safety: the current behavior refuses useful work rather than returning a known false value. It is particularly visible because the refused examples need no iterative root finder at all. The equation can be checked exactly at the already available candidate. The current helper admits this argument only when the candidate is an integer.[9]

The public-path witness is deliberately elementary:

```
s = AsymptoticInverse[x, {x, 0}, {y, 3},
    Direction -> "FromAbove"];
InverseNumericalCheck[s, 1/2, WorkingPrecision -> 50]
```

For this inverse, the exact expression is y, the source seed is $1/2$, and the original equation is $x - 1/2 = 0$. Both the source side and the target side are positive. The inspected numerical path sends that seed to the guarded helper. The integer-only exception cannot admit it; the requested precision is then above the guard's machine-precision threshold. The anticipated result is a structured precision-unavailable failure, subject to the unexecuted public construction and dispatch being reproduced as specified.[7, 8, 9]

This report does *not* claim a freshly observed wrong expansion, false error theorem, false interval enclosure, or measured package speedup. The local environment had no usable Wolfram or Mathics runtime; the available evaluator connection failed. Thirty independent Python test methods passed. Those tests establish properties of the supplied exact-arithmetic reference implementation and countermodels; they do not turn the WL patch into a validated package change.

The recommended repair is narrow: extend candidate admission, not the claimed precision of the root finder. A proposed rational candidate is accepted only if an exact polynomial equation vanishes at that exact candidate. The old precision refusal remains in force for everything not proved by this path. The accompanying WL overlay is experimental and unexecuted; the accompanying bounded Python checker and its tests were executed.

2 Scope and nonduplication

2.1 A pinned and bounded review

All repository citations resolve to `8e859961d7d37f008b826f3a8cad406460271614`. Reading a moving default branch would make it too easy to compare a new implementation against an obsolete report without noticing the mismatch. The pin identifies the source reviewed here; it is not a claim about any later version of the repository.

Inspection covered public construction and numerical dispatch, ordinary inverse machinery, selected series operations, Gamma/Barnes inverse and forward paths, source-coordinate transformations, exact interval certification, selected Dirichlet-family code, Mathics adapters, and validation documentation/tooling.

The coverage ledger records those areas and the nature of the inspection. This is substantial targeted source analysis, not a statement that every line of the generated package, every vendored article, or every historical review was read.

In particular, the novelty screen relies primarily on maintained consolidated crosswalks. Relevant originals were consulted where the summaries suggested a possible overlap: the wave-2 index specifically identifies report 15's powered-observable topic, and report 28's original article makes the previous numerical-precision finding explicit. No assertion is made that no similar sentence exists anywhere in unindexed historical prose.[5, 6]

2.2 Why N01 is not a repetition of W4-02

The earlier W4-02 issue concerns a numerical solver path being described as high precision without checking the precision actually supplied. The current code introduces a conservative refusal and a separately proved exact-integer escape route. This review preserves the safety objective of that change.[4, 6, 9]

N01 concerns the *new exception's acceptance class*. The equation $2x - 1 = 0$ at the already represented seed $x = 1/2$ can be proved exactly just as $x-1 = 0$ can be proved at $x = 1$. Refusing the former because its exact point has denominator two is not necessary to maintain the numerical safeguard. Extending that exact proof path is a concrete additional development opportunity, not another recommendation to add an achieved-precision check.

The distinction is important for issue tracking. N01 should be linked as a follow-on to the guarded implementation, with its own acceptance tests. It should not reopen the earlier false-precision claim as though the guard were absent, and it should not be counted separately for each dyadic number, source scaling, or working-precision setting.

2.3 Evidence categories

Category	Meaning in this report
Source fact	A condition or call path visible at the pinned revision.
Deduction	A mathematical or control-flow consequence, with assumptions stated.
Independent execution	The supplied Python implementation or arithmetic model was executed.
Unrun characterization	WL code supplied to establish actual evaluator and public-path behavior.
Not established	Complete integration, a native failure transcript, global absence of other bugs, or package timing.

A failure to obtain a runtime is an evidence limitation, not a package defect. Similarly, a conservative failure is not interchangeable with a wrong answer. Those distinctions constrain both the article's language and the machine-readable evidence files.

3 Architectural observations and coverage

The public entry defines a remainder-aware expansion object and dispatches into specialized modules. Numerical checking is a consumer of retained symbolic information, not the mechanism that constructs every expansion. The ordinary inverse path retains the original forward expression, variables, selected endpoint, direction, power observable, and target-domain information; the numerical consumer reconstructs a source seed, solves the stored equation, and compares the requested observable.[7, 8]

This separation explains why a missing arbitrary-precision solver need not block all high-precision checks. If the retained expression already produces an exact solution at a particular target, a numerical iteration would be redundant. A small exact-verification path can establish that particular root while leaving the general solver capability unchanged.

The Mathics numerical module is installed late and rewrites five package-owned consumers: the ordinary numerical inverse evidence function, coordinate and source-coordinate checks, Gamma/Barnes numerical checks, and the special-inverse numerical path. It does not replace caller uses of `System‘FindRoot`. A repair at the exact-candidate helper therefore has a narrow implementation surface, but acceptance must still examine the different consumer contracts.[9]

Area	Inspected source / boundary	Review disposition
Public and ordinary inverse	`AsymptoticAnalysis.wl`; construction, coefficient/residual and numerical entries	Public route for N01; no further item retained
Numerical evidence	`NumericalInverseChecks.wl`; seed, domain, root and observable	Exact information is converted to a numerical seed before the guard
Mathics numerical adapter	`MathicsNumerical.wl`; exact-integer exception and precision refusal	N01 retained
Series arithmetic	`SeriesOperations.wl`, `SeriesArithmetic.wl`, refinement requests	Relevant apparent overlaps excluded
Specialized inverse families	Gamma/Barnes checks and operations; coordinate/source-coordinate code	Inspected selected contracts; not declared defect-free
Certification	`InverseCertificates.wl`; interval operations, domain proof and scope	Branch-connectivity alarm rejected under explicit scope
Other Mathics adapters	Assumptions, Taylor, simplification, calls, compatibility, calculus and time budgets	Existing inventory used to suppress duplicates
Other mathematical paths	Selected real-domain, Barnes-forward and Dirichlet-family code	No additional supported finding retained
Validation and provenance	Maintained receipts/documentation, acceptance-checker source and review crosswalks	No receipt treated as an execution performed here

The phrase “no additional item retained” is a review result, not a proof of correctness. It can mean that a suspicion was already reported, that a guard makes the behavior conservative, that the contract intentionally excludes the proposed input, or that the available evidence was insufficient to support a fresh claim.

4 N01: exact dyadic roots are outside the integer-only exception

4.1 The decisive source condition

The helper `mathicsNumericalExactIntegerSeed` is held. It blocks the source variable, requires a real numeric seed, rounds that seed to an integer, and rejects the request unless the seed is that integer or exactly the numerical representation of that integer at the seed's precision. It then requires a held equality, forms its difference, checks an exact finite polynomial, and accepts only literal zero after substitution.[9]

The relevant predicate is:

```
candidate = Round[seed];
If[! IntegerQ[candidate] ||
   ! (SameQ[seed, candidate] ||
      SameQ[seed, N[candidate, Precision[seed]]]),
   Return[$Failed, Module]];
```

This is a sensible sufficient condition for recognizing an unchanged integer. It is not a sufficient implementation of exact *rational* candidate admission. For a real seed representing $1/2$, either integer chosen at the rounding tie differs from $1/2$. The helper returns before attempting the exact equation $2(1/2) - 1 = 0$.

Its caller then checks whether the requested precision goal exceeds its available machine-precision threshold. If so, it throws `MathicsNumericalPrecisionUnavailable`. The exact-integer exception is evaluated before this refusal. This ordering means that extending the exception can recover a useful class without weakening the refusal.[9]

4.2 Public identity and affine witnesses

For $f(x) = x$ on the branch $x \to 0^+$, the inverse is exactly $g(y) = y$. A request at $y = 1/2$ produces an exact mathematical seed $1/2$, and arbitrary requested display precision does not create a need to approximate an unknown root. The existing numerical consumer computes the seed from the stored expression and forwards a numerical version to the helper.[8]

The integer control $y = 1$ is important. The distinction is not that one target requires a difficult asymptotic approximation and the other is easy. Both are exact instances of the same inverse. The helper's point class, not the mathematical difficulty of inversion, separates them.

An equivalent affine formulation is

$$f(x) = 2x, \qquad y = 1, \qquad g(1) = \frac{1}{2}.$$

The target is now an integer, but the source root is not. Therefore simply treating integer targets specially would repair the wrong boundary. The exact proof must concern the recovered source candidate and the retained source equation.

More generally, let

$$f(x) = ax + b, \quad a \in \mathbb{Q} \setminus \{0\}, \quad c \in \mathbb{Q}, \quad t = ac + b.$$

Then $f(c) - t = 0$ is an exact identity. Subject to the selected source/target domains, changing a or b does not change the availability of that equality proof. A future exact-root path should not make acceptance depend unnecessarily on whether c happens to have denominator one.

4.3 An infinite family and a non-root control

For integers m and $k \geq 1$, consider

$$p_{m,k}(x) = 2^k x - m, \qquad c_{m,k} = \frac{m}{2^k}.$$

Every candidate satisfies $p_{m,k}(c_{m,k}) = 0$. When the reduced denominator exceeds one, an integer-only candidate gate cannot use this identity. Dyadic values are particularly useful probes because they have finite binary representations; they avoid making the first witness depend on reconstruction of a nonterminating binary fraction.

The counter-control is just as essential. For any nonzero rational ε,

$$2\left(\frac{1}{2} + \varepsilon\right) - 1 = 2\varepsilon \neq 0.$$

A candidate that is close to $1/2$ is not thereby an exact root. The supplied tests include the next binary floating-point number above $1/2$ and an explicitly resolved small perturbation. They must not be promoted merely because their residuals are small.

4.4 Severity and limits

This is an **avoidable conservative refusal**, not evidence of unsound interval arithmetic or fabricated digits. The current source is explicit about its limited exception. The finding is that the accepted class is unnecessarily narrow for the project's compatibility objective and for exact polynomial checks; the documentation is not accused of promising rational-root completeness.[9, 2]

The simplest current manifestation is predicted at the public numerical-check API. Until the supplied WL probes run against the exact pin, the report does not claim an observed failure tag, a passed integration test, or a measured improvement. The private source predicate itself is directly inspectable and its integer-versus-dyadic distinction does not require a numerical experiment.

5 A proof-preserving repair

5.1 Separate proposal, proof, and output

The repair has three logically different steps. First obtain a candidate point. Next prove that the retained exact equation vanishes there. Only then format the proved point numerically at the requested precision. Neither candidate reconstruction nor formatting is the proof.

For an exact rational polynomial p and exact rational candidate c, the acceptance condition is simply

$$p(c) = 0 \quad \text{in exact arithmetic.} \tag{1}$$

No positive residual tolerance is used. The existing domain and source-side checks remain consumers of the resulting root. Equation (1) alone does not prove uniqueness, membership in a particular inverse germ, or validity of an unspecified input-remainder family.

Proposition 5.1 (Safe exact admission)**.** *Let $p \in \mathbb{Q}[x]$ be the exact retained equation and $c \in \mathbb{Q}$ a proposed candidate. An algorithm that accepts only after verifying $p(c) = 0$ by exact arithmetic cannot accept a point that fails that equation, regardless of how the candidate was proposed.*

Proof. Acceptance includes a decision that the rational number $p(c)$ equals the rational number zero. Rational arithmetic is exact; therefore the equality holds. Candidate generation has no role in this implication. It can affect completeness or cost, but not this equality conclusion. □

The word "safe" here is deliberately local. It refers to the equation equality, not to a theorem about a mutable evaluator, arbitrary callbacks, all source domains, or global branch identity. The WL implementation must still preserve evaluation semantics and exactness checks before the elementary mathematical argument applies.

5.2 Candidate reconstruction is not exactification of an answer

Official Wolfram documentation distinguishes a rational reconstruction compatible with an inexact number's precision from the exact rational encoded by its stored bits. In particular, `Rationalize[x,0]` is not documented as identical to a bitwise extraction in every case. The supplied proposal therefore treats its output as a *candidate*, validates the point independently, and keeps a round-trip identity screen.[11]

The implementation must not use `SetPrecision` to manufacture trustworthy digits of an unproved approximate root. Exact numbers have infinite precision in Wolfram's representation, but turning a float into an exact rational only specifies a point; it does not prove that the point solves the original equation.[12]

For example, the exact rational represented by a Python binary float approximating $1/3$ is generally not $1/3$. The independent test checks that it fails $3x - 1 = 0$ exactly. The separate test with the original rational candidate $1/3$ succeeds. These are intentionally different tests, and the Python conversion is not described as an emulator of Wolfram's `Rationalize`.

5.3 The supplied WL overlay

`code/ExactSeedCandidate.wl` replaces one package-private helper in a disposable Mathics session after loading the package. It preserves the legacy private helper name so the five already-bound consumers do not need a new rewrite. It retains the original integer shortcut and adds a bounded rational-candidate attempt when the shortcut does not apply.

The added path requires a finite real numeric seed, a rational candidate, a round trip to the seed at its recorded precision, and manageable reconstruction input/point size. The retained equation must still be an exact finite polynomial, and direct substitution must still produce literal zero. The original general precision refusal remains unchanged. The overlay does not alter `System‘FindRoot`, native Wolfram dispatch, the series constructor, or the source/target domain checks.

The overlay uses fully qualified package symbols rather than relying on a late `Begin` nested inside a compound expression to retokenize names. Its local early exits use an explicit catch tag rather than assuming that a late-loaded `Return` binds to the same compatibility wrapper as the original source. These are implementation precautions for this candidate, not additional repository findings.

The WL file has **not been executed**. Its reconstruction bounds are candidate policy, not a claimed complete resource bound for the package's polynomial evaluator. It is not a production patch to install automatically in a working research session. Baseline and candidate behavior need separate fresh processes, and the final source change should be made in the maintained modular source and regenerated through the repository's normal workflow after acceptance.

5.4 Preserve the exact seed before numerical conversion

A fuller implementation should first inspect the exact expression after exact target substitution, before calling `N`. That exact candidate is already available for examples such as $g(1) = 1/3$ when $f(x) = 3x$. Recovering it from a rounded numerical representation is unnecessary.

This is a local extension of N01's remedy, not a proposal to turn the whole package into an exact solver. An internal record can distinguish an original exact candidate from a reconstructed candidate and a merely numerical seed. The equality check remains identical. If no exact candidate is proved, the current numerical capability guard handles the request as before.

The first implementation should remain narrow: rational polynomial equations and rational candidates. Algebraic candidates, transcendental identities, numerical black boxes, and generalized interval oracles require additional contracts and are outside the delivered implementation. No completeness claim for exact roots follows from admitting rationals.

6 A small exact checker with explicit work limits

6.1 Clearing denominators

The Python companion is independently implemented rather than a translation of the full package. It accepts coefficients in increasing degree order and an exact rational candidate. Let

$$p(x) = \sum_{i=0}^{d} a_i x^i, \qquad a_i \in \mathbb{Q}, \qquad c = \frac{r}{s}, \quad s > 0.$$

Choose the positive common denominator

$$D = \operatorname{lcm}_{0 \le i \le d} \operatorname{den}(a_i), \qquad A_i = D a_i \in \mathbb{Z}.$$

Then define the homogeneous integer evaluation

$$H(r, s) = \sum_{i=0}^{d} A_i r^i s^{d-i}. \tag{2}$$

Lemma 6.1. *For the data above, $H(r, s) = D s^d p(r/s)$. In particular, $p(c) = 0$ if and only if $H(r, s) = 0$.*

Proof. Multiply each term $a_i (r/s)^i$ by $D s^d$ and use $D a_i = A_i$. The multiplier $D s^d$ is nonzero, so it preserves equivalence to zero. □

The checker performs only integer arithmetic after clearing denominators. It returns an equality witness containing the exact input polynomial and candidate. The witness explicitly says that it does not establish uniqueness, inverse-branch identity, or package acceptance.

6.2 Linear-length homogeneous Horner evaluation

Computing every power independently is unnecessary. Initialize $v = A_d$ and $q = 1$. Moving downward through the remaining coefficients, update

$$q \leftarrow q s, \qquad v \leftarrow v r + A_i q.$$

After the final coefficient, $v = H(r, s)$. Inductively, after j steps,

$$v = \sum_{k=0}^{j} A_{d-k} r^{j-k} s^k.$$

The loop uses a number of integer arithmetic operations linear in the supplied degree. That is not a unit-cost or wall-clock claim: integer lengths grow with degree and with the sizes of r, s, and the coefficients.

The implementation uses coefficient-count, input-bit-size, and predicted intermediate-bit-size limits. It checks a conservative bound before constructing the potentially large cleared-denominator evaluation. Unknown or over-budget cases decline the exact-admission attempt; they are not silently accepted and no coefficient is dropped.

6.3 A one-sided modular filter

The reference implementation also evaluates H modulo two small positive moduli, 101 and 103. A nonzero modular residue proves that $H \neq 0$, so it can cheaply reject many incorrect candidates. Passing both modular filters does *not* establish equality. The exact integer computation is still mandatory on the acceptance path.

The polynomial $p(x) = 101 \cdot 103$ is a useful negative control: both modular residues vanish for every candidate, but the polynomial is never zero. The tests require rejection after the exact stage. The homogeneous form does not divide by the candidate denominator modulo either modulus, so candidates whose denominators are divisible by 101 or 103 remain valid test cases.

These performance ideas apply only to the narrow independent rational-polynomial checker. No claim is made that this loop is faster than the repository's evaluator, that its modular filter always pays for itself, or that a package benchmark has been run. In small affine examples, retaining the original exact candidate is likely more important than adding a sophisticated filter; that is an engineering expectation, not a measured result.

6.4 Verifiable receipts rather than trusted flags

The point-equality examples in `evidence/point-equality-examples.json` can be rechecked by the supplied verifier. It recomputes the polynomial equality instead of trusting stored degree, denominator, or zero-value fields. It rejects altered candidates and inconsistent metadata in the tests.

This receipt format is intentionally not a replacement for the repository's validation receipts. It proves an equation about the explicitly included polynomial. It does not authenticate a repository revision, bind mutable user definitions, certify source domains, or demonstrate that an interpreter executed the accompanying WL program.

7 Validation performed and remaining acceptance

7.1 Executed independent checks

The independent suite ran on Python 3.13.5 on Linux. All **30 unittest methods passed**, with zero errors and zero failures. The supplied transcript and JSON receipt record that population. Nested cases are reported separately: 96 dyadic cases, nine affine rescalings, 200 deterministic rational-oracle

comparisons, and 150 constructed-root cases. These nested counts are not added to a fictional package acceptance total.

The tests exercise the following distinct obligations:

- The integer-only model rejects noninteger dyadics, while exact rational equality accepts them; integer controls are unchanged.
- A neighboring float, an approximate irrational root, and the exact binary value of an approximation to 1/3 do not pass the wrong polynomial equation.
- Cleared-denominator evaluation agrees with independent `Fraction` Horner evaluation on deterministic random inputs; constructed polynomial roots pass.
- Modular false positives, denominator/modulus interactions, tampered receipts, inexact input, nonfinite seeds, and work-limit refusals behave as specified.

These are tests of the mathematical repair idea and its Python reference code. The exact model of the old gate operates on rational seeds; it is not an implementation of Mathics precision metadata, its parser, the numerical solver, or the entire package.

7.2 WL characterization and regression files

`code/Probe.wl` emits raw observations from a fresh interpreter. It records the runtime string, package path, private-helper controls on Mathics, and public numerical-check cases. It deliberately does not print a passing acceptance summary merely because it reached the end.

`code/public_regressions.wlt` contains desired-contract MUnit tests for the public API. It checks the integer control, 1/2, 3/8, affine source scaling, the target-side refusal, and preservation of a source condition excluding the candidate. The tests are unrun. Mathics execution requires a runner that actually supports the chosen test format; supplying a `.wlt` file does not establish that support.

Case	Expected current/candidate distinction	Required evidence
Integer exact root	Both should preserve the exact shortcut	Same numerical result and domain checks
Dyadic exact root	Current integer gate declines; candidate should prove it	Exact root equality and requested output representation
Resolved nearby non-root	Neither may promote it	Exact rejection, not a residual-tolerance decision
Inexact equation	Exact shortcut remains unavailable	No conversion of approximate coefficients into a theorem
Nonpolynomial equation	Existing general guard remains applicable	No accidental new transcendental proof path
Excluded source point	A proved equation root must still fail the domain check	Public refusal survives candidate admission
Non-dyadic exact expression	Exact seed preservation may help; reconstruction alone need not	Separate original-exact and reconstructed-candidate records
Native Wolfram run	Mathics-only overlay must leave it unchanged	Fresh native controls, not inferred preservation

7.3 What is not established

There is no successful native or Mathics package run in this review, no current package timing, no demonstrated full-checkout installation of the overlay, and no proof that every relevant historical sentence was searched. The code and mathematical argument reduce the amount of work needed to validate N01; they do not eliminate evaluator-level acceptance.

The independent test runner regenerates its own evidence files. It does not invoke the repository, a remote kernel, a package test suite, or a network service. This makes its successful result reproducible without confusing its scope with the unexecuted integration work.

8 A certificate suspicion that did not survive contract checking

A potentially serious-looking counterexample arose during the audit: an interval can isolate a root on the same source side as an inverse expansion without identifying the root belonging to that expansion's local germ. On reading the complete certificate result construction, however, the source explicitly limits its claim to a unique root of the stored function in the verification interval, on the selected side and within retained source conditions. It expressly does not assert a global inverse-branch certificate.[10]

For example,

$$f(x) = x - 2x^2, \qquad t = \frac{9}{200}$$

has positive roots $1/20$ and $9/20$. The inverse germ at $x = 0^+$ selects the smaller root as $t \to 0^+$. A small interval around $9/20$ can nevertheless be a valid unique-root enclosure for the stored equation. A certificate making only the stated interval claim is not false merely because a reader hoped it would certify the germ-selected root.

The distinction cannot be repaired conceptually by checking only the derivative's sign. For

$$f(x) = x(x-1)(x-2), \qquad t = \frac{231}{1000},$$

the three roots are

$$\frac{9-\sqrt{37}}{20}, \quad \frac{9+\sqrt{37}}{20}, \quad \frac{21}{10}.$$

The first and third roots have positive derivative, but only the first is connected to the 0^+ inverse germ at small positive target. These examples explain why exact point equality, local interval uniqueness, and branch connectivity are different propositions.

Disposition: not counted as a new defect or a new certification recommendation. The code's explicit scope prevents the proposed example from establishing the suspected false theorem. The independent point-equality checker similarly avoids a branch-identity claim. This is included to document a rejected alarm, not to inflate the finding count or restate the existing certification roadmap.

A second apparent powered-observable coverage issue was excluded at the novelty stage because the wave-2 index explicitly identifies report 15 as covering it. Other familiar routes were likewise left to their existing work items.[5]

9 Focused development sequence

Only development work directly needed to resolve N01 is recommended here. Broader package architecture, native compatibility, precision propagation, review infrastructure, and mathematical-extension recommendations remain in the pre-existing reports rather than being copied into this article.

9.1 First: characterize the new acceptance boundary

Run the integer, dyadic, nearby-non-root, and source-domain controls against the pinned baseline in a fresh Mathics process. Save the actual outputs and runtime version. Then run the public identity and affine cases. A private helper failure establishes its local behavior; it does not substitute for showing that the public call reaches it.

The success condition is not merely an absence of messages. A positive case must return the correct reference root or the desired numerical-check association, and a negative case must preserve its refusal. The source-domain case is especially important because exact equation satisfaction alone is weaker than all of the public call's requirements.

9.2 Second: integrate the narrow rational path

Implement exact candidate admission without changing the general solver's advertised capabilities. Start with retained rational candidates and exact polynomial equations. Retain the existing integer controls, precision refusal, original equation, and post-root domain checks. Evaluate the experimental overlay as a candidate, not as an automatically trusted hotfix.

Avoid broad rational guessing as the acceptance test. A reconstruction policy may propose a point, but only equation (1) permits its exact admission. A failed or over-budget proof attempt must leave the conservative behavior intact rather than reducing the requested precision silently.

9.3 Third: retain pre-numerical exact information

Add an internal exact-candidate slot at the point where the stored expression is specialized to an exact target. This avoids unnecessarily losing 1/3 before a solver is even considered. The slot must not be a second unsynchronized copy of the entire expansion; it is request-local evidence derived from the already retained expression and target.

Keep the distinction visible in numerical evidence: an exact equation-verified candidate, an approximate solver result, and an unsupported precision request are different outcomes. That distinction explains why one exact point can be available at a requested precision even when the general root finder is not.

9.4 Acceptance, not a completeness claim

The narrow change is ready only after its selected public and private cases pass in supported Mathics and native configurations. It does not establish arbitrary-precision Mathics root finding, exact solution of all polynomial equations, algebraic-root completeness, branch connectivity, or complete compatibility across every exported API.

This bounded endpoint is useful precisely because it is achievable and testable without weakening the existing safeguard or reopening the previous review inventory.

10 Using the archive

The bundle is self-contained for the independent tests and the article build. From its root directory:

```
python code/run_reference_tests.py
cd article
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
```

The Python test runner requires only the standard library. It rewrites the independent receipt files; preserve a copy of the delivered evidence before regenerating them when comparing environments. The WL characterization script expects `$AuditPackagePath` to be set to an existing local package entry. The optional `$AuditPatchPath` selects the experimental overlay. Use separate fresh sessions for baseline and candidate observations.

Artifact	Purpose
`article/review.tex`, `article/review.pdf`	Article source and compiled version
`code/exact_seed_reference.py`	Bounded independent rational-polynomial equality checker
`code/test_exact_seed.py`	Independent mathematical and admission tests
`code/ExactSeedCandidate.wl`	Unexecuted Mathics-only private-helper overlay
`code/Probe.wl`	Unexecuted raw interpreter characterizations
`code/public_regressions.wlt`	Unexecuted desired public contracts
`evidence/independent-tests.json`	Executed test count and explicit scope
`evidence/novelty-ledger.json`	Retained item, overlap boundary, excluded alarms
`evidence/review-scope.json`	Pin, runtime limits and source coverage

11 Conclusion

The additional result of this review is modest but concrete. A new safety guard has a useful exact-root escape route, yet unnecessarily ties that route to integer source values. The same equality proof can cover rational points without trusting an inadequate numerical solver. The report isolates that boundary, supplies an infinite family of exact witnesses, derives a bounded verification algorithm, and tests an independent implementation.

The remaining step is evaluator-level acceptance of the candidate and the public witnesses. Until then, the correct description is a source-supported compatibility opportunity with executed independent mathematics, not a native-verified package fix. Existing findings are left to their original reports, and an apparent certificate counterexample is rejected because it does not contradict the code's explicit claim.

References

[1] Asymptotic contributors. Code review index, pinned revision. Four-wave inventory and evidence distinctions.

[2] Asymptotic contributors. Code review implementation status, pinned revision. Consolidated earlier findings and coverage targets.

[3] Asymptotic contributors. Wave 3 intake, pinned revision.

[4] Asymptotic contributors. Wave 4 intake, pinned revision. See W4-02 and the distinction between incoming reports and later implementation.

[5] Asymptotic contributors. Wave 2 review index, pinned revision. Original report topics and scope.

[6] Report 28. Numerical contracts, Mathics semantics, and validation infrastructure, original article preserved in the repository. Its F01 concerns the earlier unguarded precision boundary.

[7] Asymptotic contributors. Public kernel and ordinary inverse implementation, pinned revision. See public usage, inverse construction, and the numerical dispatch in `InverseNumericalCheck`.

[8] Asymptotic contributors. Numerical inverse checks, pinned revision. In particular seed construction, root solving, source-domain validation, and observable comparison.

[9] Asymptotic contributors. Mathics numerical admission adapter, pinned revision. Lines 10–22: exact-integer helper; lines 24–45: precision guard; final lines: five consumer rewrites.

[10] Asymptotic contributors. Inverse certificates, pinned revision. See `certAttempt`, result metadata, and its explicit scope statement.

[11] Wolfram Research. Rationalize documentation. Consulted September 10, 2026. Distinguishes precision-compatible rational reconstruction from bitwise rational extraction.

[12] Wolfram Research. Precision documentation. Consulted September 10, 2026. Exact numbers, approximate numbers, and precision semantics.

[13] Wolfram Research. FindRoot documentation. Consulted September 10, 2026. Numerical solver and precision-goal semantics; not a specification of Mathics behavior.